SD242 – Verificação de Sistemas Digitais (80h) UVM

Autores	
Anderson Max Almeida e Porfirio (Hbr)	Nome 2 (INSTITUIÇÃO)

Histórico de revisões		
05/11/2025	V1.0	Versão inicial

Tópicos

- Introdução
- UVM
- Componentes Principais
- UVM Components
- Standard UVM Phases
- Build Phases
- Run Time Phases

Página em branco.

Aula 8
Introdução a UVM

Introdução

- A rápida evolução dos sistemas de CIs entre os anos 90 e 2000, exigia melhores métodos de verificação.
- VHDL e Verilog eram as principais linguagens de programação HDL, que também eram usadas para verificar os projetos.
- O consórcio Accellera assumiu a tarefa desenvolver linguagem de Descrição e Verificação de Hardware.
- Desta necessidade de que surgiu o SystemVerilog.

Introdução

- O SystemVerilog introduz a programação orientada a objetos que facilitam a criação de testbenches complexos, modulares e reutilizáveis, para verificar o design do hardware.

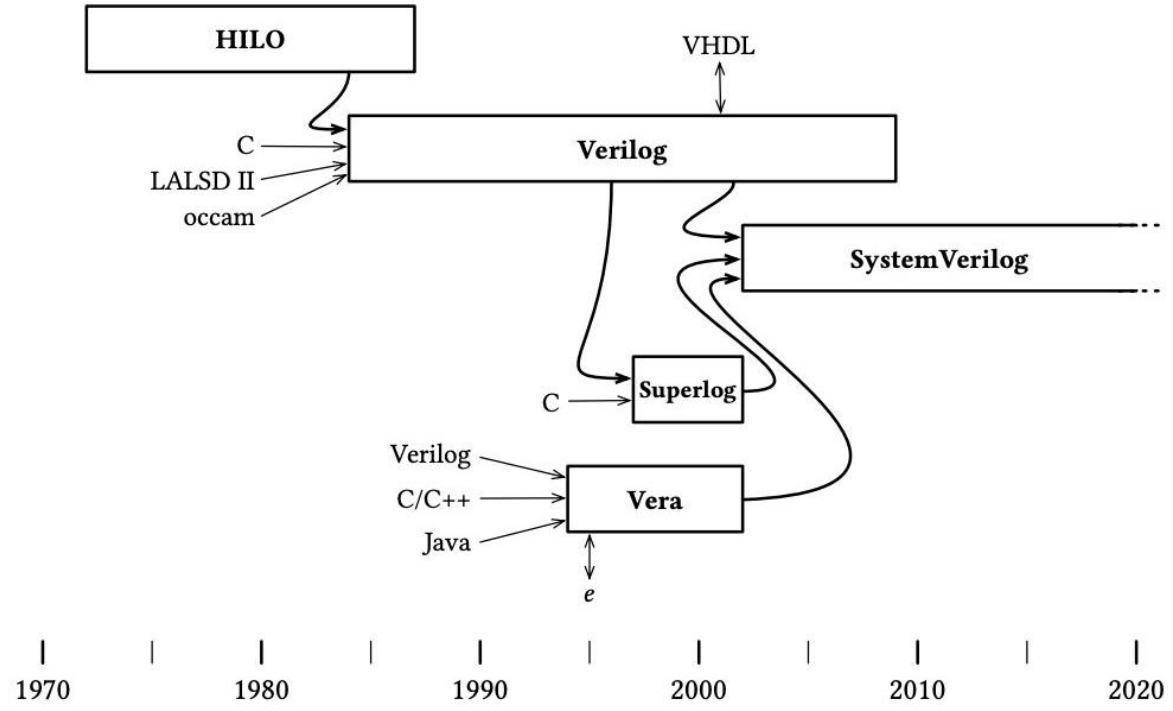


Fig. 1. The Verilog family tree, 1972–2020 and beyond.

Introdução

- "e" Reuse Methodology; Reuse Verification Methodology; SystemC (TLM)

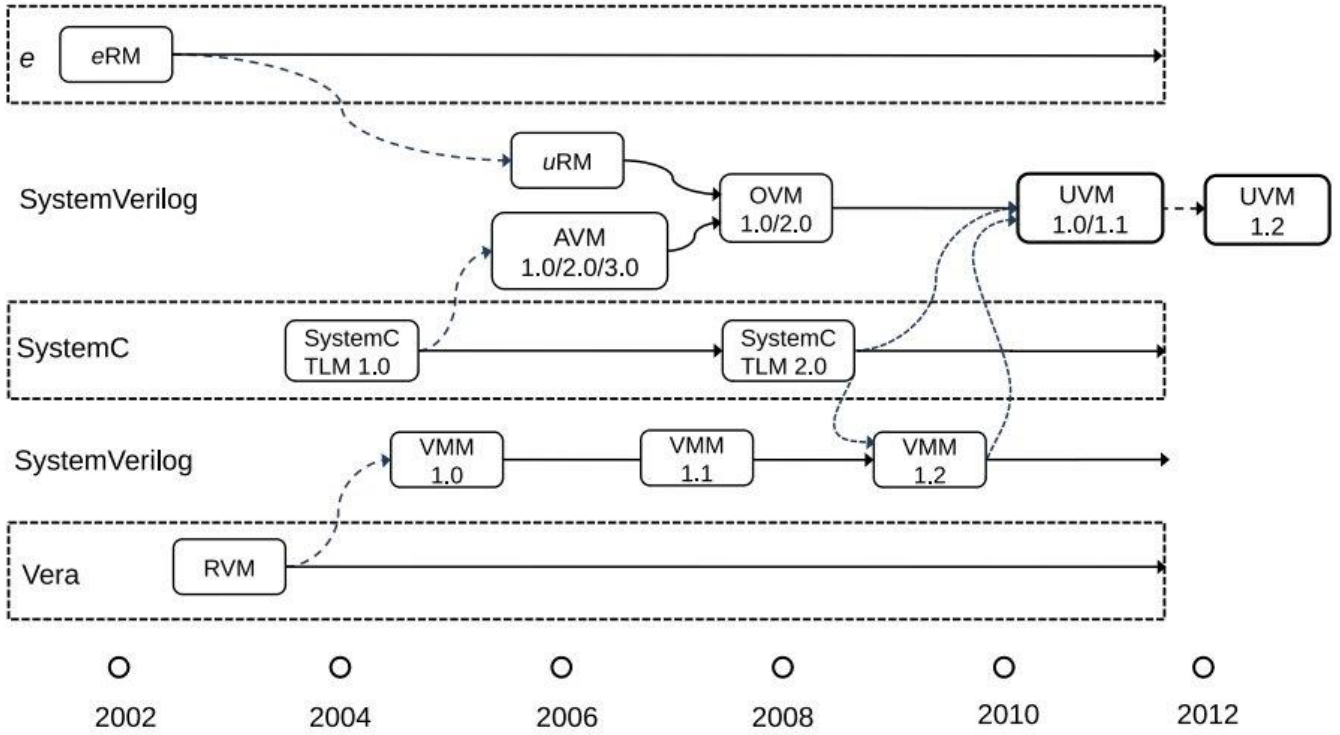


Fig. 18. Family tree for UVM.

UVM (Universal Verification Methodology)

- É uma metodologia de verificação de hardware que utiliza uma abordagem orientada a objetos e em camadas.
  - Camada de Interface
  - Camada de Transação

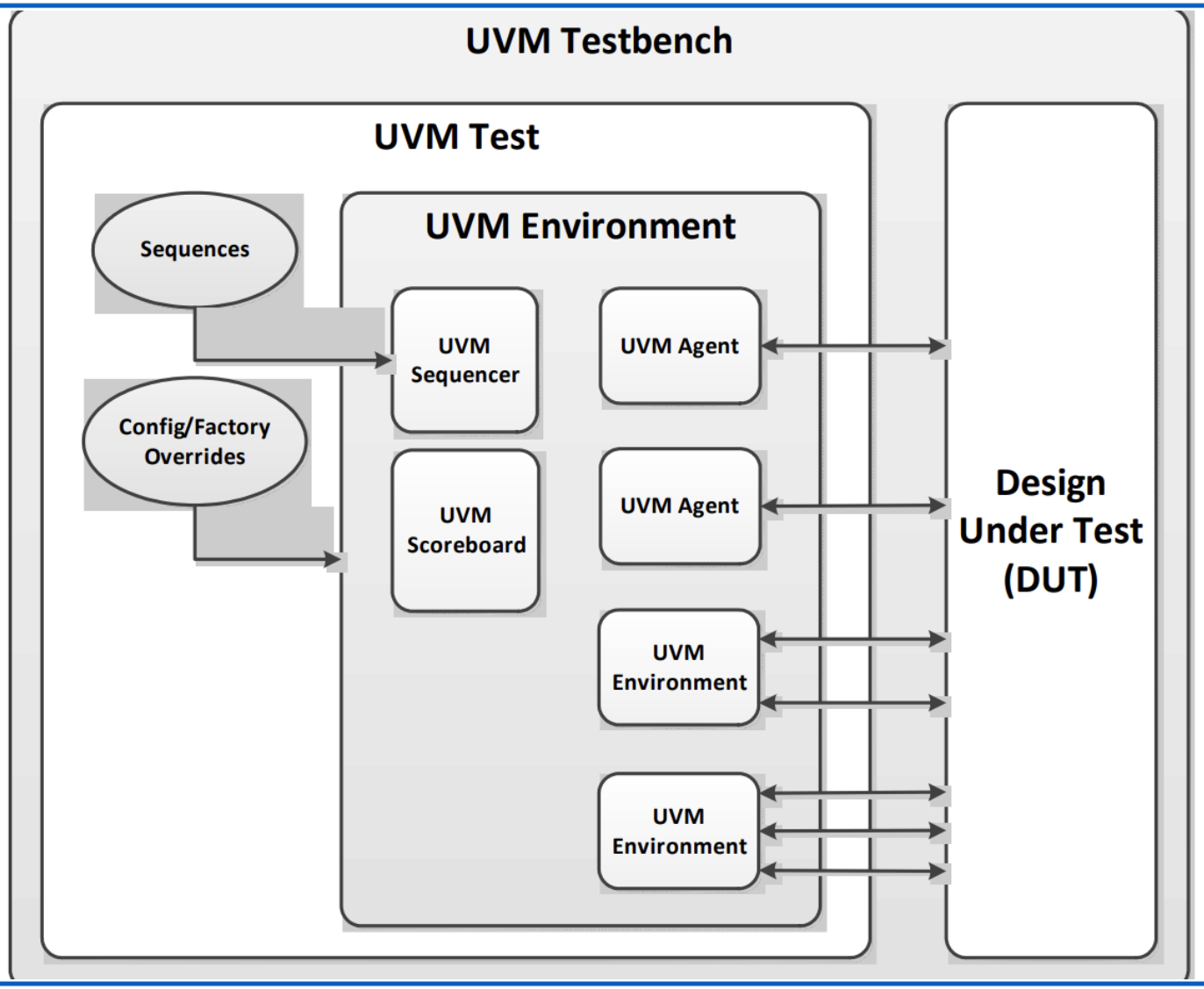
- Separação de escopos, permitindo que diferentes desenvolvedores foquem em suas tarefas.
  - Aumenta Produtividade
  - Trazendo Modularidade
  - E permite Reutilização

UVM (Universal Verification Methodology)

- Camada de Interface (Pinos)
  - Camada mais baixa, conectada diretamente à DUT.
  - Inclui Drivers (convertem transações em sinais/pinos)
  - e Monitors (observam sinais/pinos e convertem em transações).

- Camada de Transação
  - Camada superior onde a lógica de verificação opera.
  - Scoreboards (verificação do design).
  - Coverage Collectors (cobertura funcional).
  - Sequencers (geração de estímulos).

UVM (Universal Verification Methodology)



Componentes Principais

- uvm_test (Teste)
  - Instancia e configura o uvm_env.
  - Escolhe as sequências de estímulo.
  - Configura componentes via objetos de configuração.
  - É iniciado pela chamada run_test() no módulo top-level.

- uvm_env (Ambiente)
  - Container que encapsula todos os uvm_agents e outros componentes específicos do design.
  - Pode ser hierárquico para reutilização em diferentes níveis (bloco, subsistema, sistema).

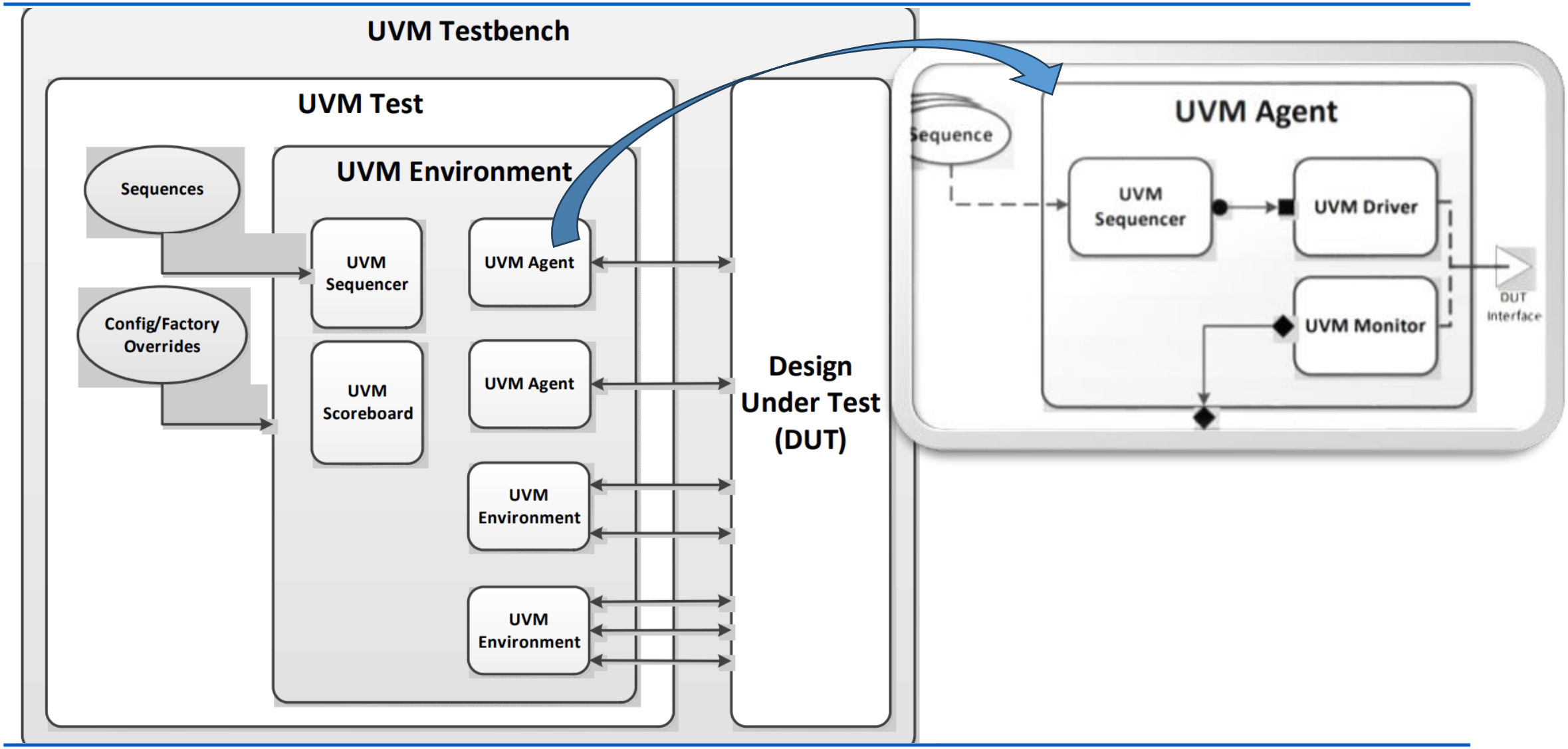
Componentes Principais

- uvm_agent
  - Bloco/componente de cada interface do DUT.
  - Isola os detalhes do protocolo.
  - Modo ativo e modo Passivo.
  - Agrupa os componentes necessários para um protocolo.
    - Driver (Objeto para sinais)
    - Monitor (Sinais para objetos)
    - Sequencer (Controla sequences)
    - e (opcionalmente) Coverage Collectors.

Componentes Principais

- uvm_scorebord
  - Responsável por verificar o funcionamento correto do DUT.
  - Ele recebe as transações de entrada, calcula/preve qual deveria ser a saída correta usando um modelo de referência (golden model) ou algoritmo conhecido.
  - Compara a saída real do DUT com a saída esperada que ele previu.
  - Valida se o DUT está se comportando conforme esperado.

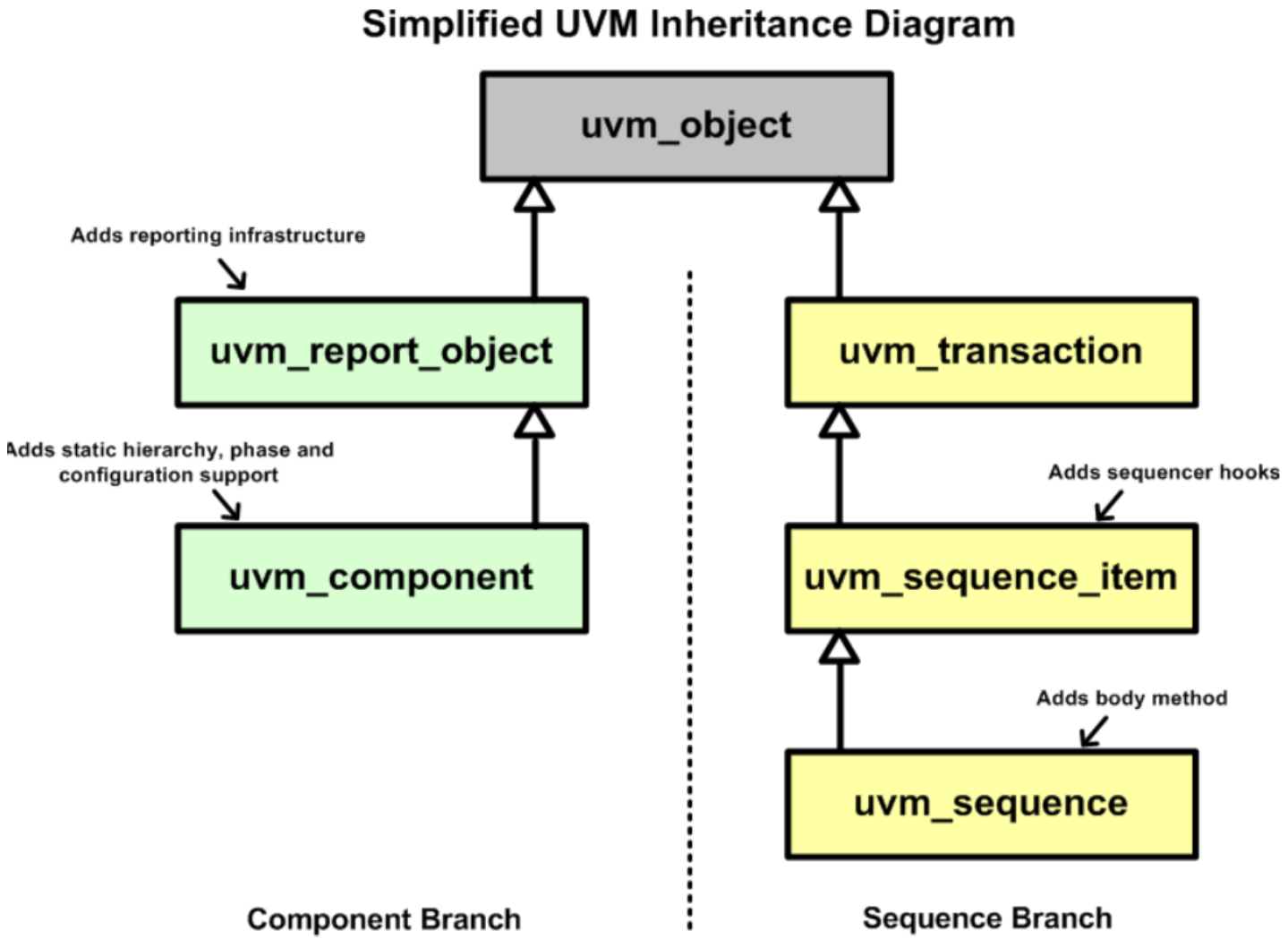
UVM (Universal Verification Methodology)



UVM Components

- Um testbench UVM é formado por componentes derivados de uvm_component, que constroem uma hierarquia estática ativa durante toda a simulação.
- Essa hierarquia é usada para mensagens de relatório, configuração de componentes e substituições pela UVM Factory, permitindo alterar componentes sem modificar o código-fonte.
- Em contraste com o *sequences* da hierarquia de classes UVM, que envolve objetos temporários — criados, usados e destruídos (ou coletados como *garbage*) quando não são mais referenciados.

Componentes Principais



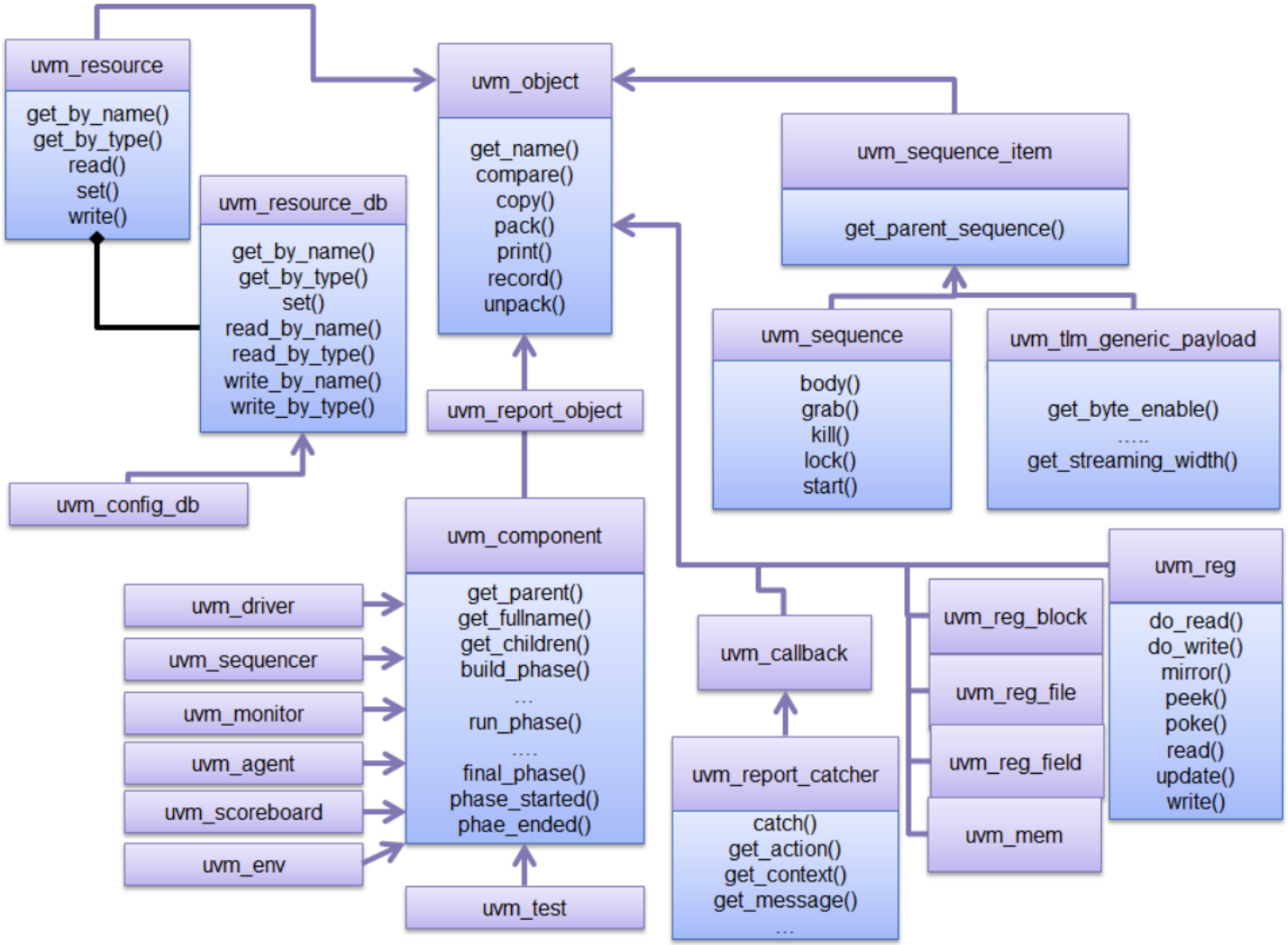
UVM Components

- uvm_object
  - A principal função é definir um conjunto de funções utilitárias comuns, como imprimir, copiar, comparar e registrar, que podem ser utilizadas por qualquer outra classe em um ambiente de teste UVM.
- uvm_report_object
  - Fornece uma interface para o recurso de geração de relatórios UVM.
- uvm_component
  - Utiliza o mecanismo de fases para executar tarefas de inicialização e configuração em uma ordem específica.
  - As fases permitem que os componentes construam sua hierarquia interna, conectem-se a outros componentes e finalizem suas configurações antes do início do teste.

UVM Components

- uvm_transaction
  - Agrupa os sinais e informações de uma operação, como endereço e dados, para serem enviados a um design under test (DUT).
  - É obsoleta e os devem criar suas próprias classes de transação estendendo a classe uvm_sequence_item que deriva da uvm_transaction.
- uvm_sequence_item
  - Os objetos de dados que precisam ser enviados para o DUT(através do Driver).
- uvm_sequence
  - Cria e organiza objetos de uvm_sequence_item para formar cenários de teste.
  - Ela controla a ordem e o fluxo de transações, enviando-as ao sequencer e ao driver.

UVM Components

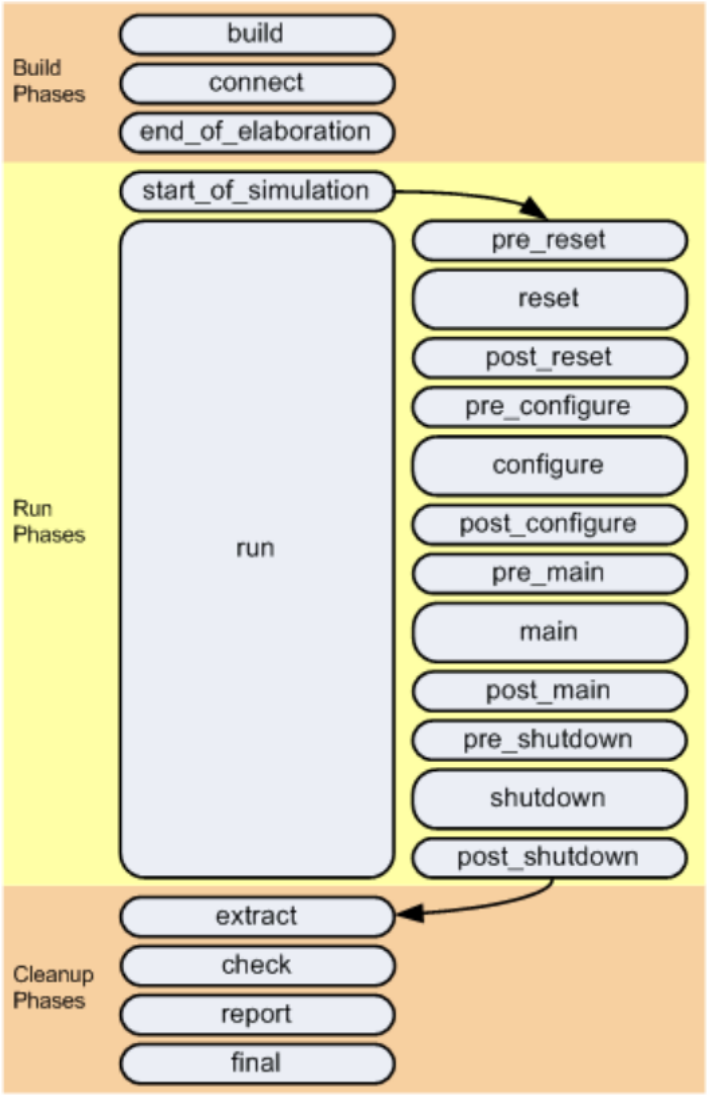


Standard UVM Phases

- Para ter um fluxo de execução de testbench consistente, a UVM usa fases para ordenar os principais passos que ocorrem durante a simulação.
  - Fases de Construção (Build)
  - Fases de Execução (Run-time)
  - Fases de Finalização (Clean up)

- Garantem ordem e consistência.
- Permitem interoperabilidade entre componentes.

Standard UVM Phases



Build Phases

- As fases de construção são executadas no início da simulação do testbench UVM e seu propósito geral é construir, configurar e conectar a hierarquia de componentes do testbench.
  - build: Cria hierarquia TOP-DOWN via Factory
  - connect: Conexões TLM BOTTOM-UP
  - end_of_elaboration: Ajustes finais

- Todas são FUNÇÕES (tempo de simulação zero)

Run Time Phases

- O estímulo do testbench é gerado e executado durante as fases de tempo de execução que seguem as fases de construção.
- Após a fase start_of_simulation, a UVM executa a fase run e as fases pre_reset até post_shutdown em paralelo.

Run

(pre e post)

Reset

Configure

Main

Shutdown

Run Time Phases

- Run
  - A fase run ocorre após a fase start_of_simulation e é usada para as atividades de geração de estímulo e verificação do testbench.
  - A fase run é implementada como uma task, e todas as tasks run_phase() dos uvm_component são executadas em paralelo.
  - Transactors como drivers e monitors usarão quase sempre esta fase.
  - Ela é executada automaticamente pelo scheduler de fases do UVM para todos os componentes.

  - Classes derivadas de uvm_object são objetos dinâmicos e não hierárquicos. Não possuem a run_phase.

Run Time Phases

- Run

```
class my_test extends uvm_test;
  `uvm_component_utils(my_test)
  my_env env;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = my_env::type_id::create("env", this); // new
  endfunction

  task run_phase(uvm_phase phase);
    my_sequence seq;
    seq = my_sequence::type_id::create("seq");
    phase.raise_objection(this);
    seq.start(env.agt.seqr);
    #20;
    phase.drop_objection(this); // encerra o teste
  endtask
endclass
```

```
class my_agent extends uvm_agent;
  `uvm_component_utils(my_agent)

  my_driver drv;
  my_monitor mon;
  uvm_sequencer #(my_seq_item) seqr;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    drv  = my_driver::type_id::create("drv", this);
    mon  = my_monitor::type_id::create("mon", this);
    seqr =
   uvm_sequencer#(my_seq_item)::type_id::create("seqr
   ", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    drv.seq_item_port.connect(seqr.seq_item_export);
  endfunction
endclass
```

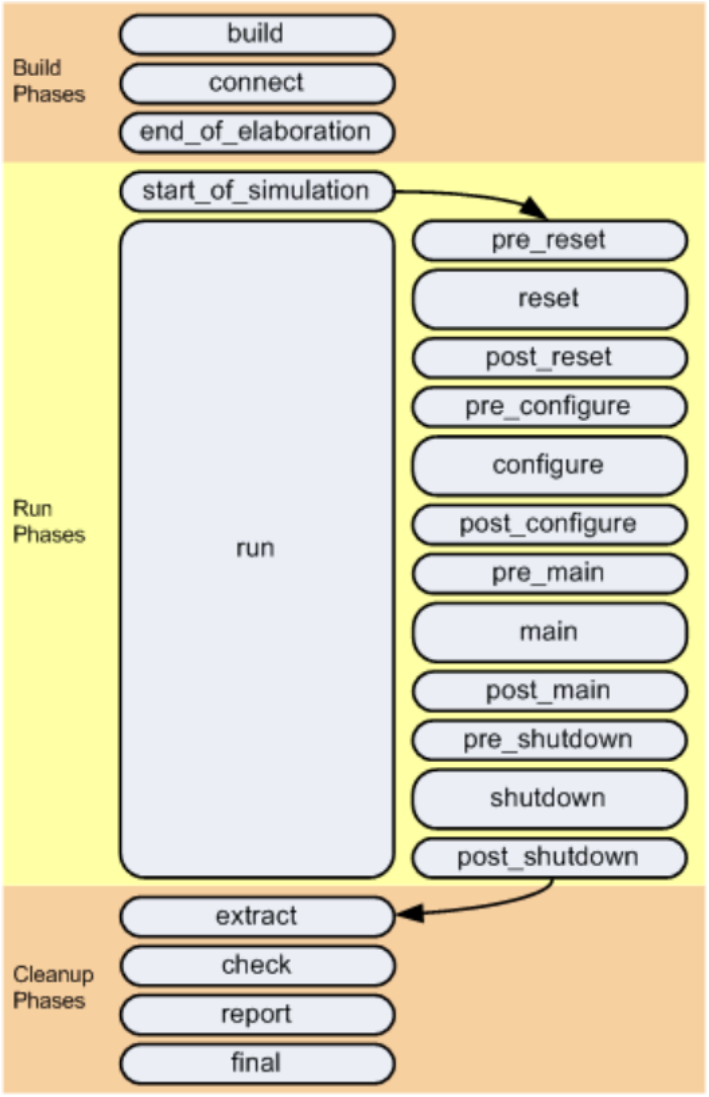
Run Time Phases

- Reset
  - Reset de DUT ou Interfaces específicos.
  - pre e pos
- Configure
  - Programar a DUT ou memórias para estarem prontas para o teste.
  - pre pos
- Main
  - Esta é a fase onde o estímulo do teste é gerado e aplicado à DUT.
  - Ela termina quando todo o estímulo é esgotado ou um tempo limite ocorre (timeout).
  - pre e pos
- Shutdown
  - É usada para garantir que os efeitos do estímulo gerado durante a fase principal tenham se propagado através do DUT e coletados.
  - Ela também pode ser usada para executar sequências que leem registros de status.

Clean Up Phases

- As fases de finalização são usadas para extrair informações de scoreboards e monitores de cobertura funcional. Observa se o caso de teste foi PASS e/ou atingiu suas metas de cobertura.
- extract
  - É usada para recuperar e processar informações de scoreboards e monitores de cobertura funcional. Isso pode incluir o cálculo de informações estatísticas usadas pela fase report.
- check
  - É usada para verificar se o DUT se comportou corretamente e para identificar erros que possam ter ocorrido durante a execução do testbench.
- report
  - É usada para exibir os resultados da simulação ou para os gravar em um arquivo.
- final
  - É usada para concluir quaisquer outras ações pendentes que o testbench ainda não tenha finalizado.

Standard UVM Phases



Laboratório

- Monte seu primeiro ambiente uvm usando o arquivo exemplo.
- Crie um pacote com as classes uvm e pesquise onde que ele deve ser importado.
- Crie um files.f a ser chamado na linha de comando do teste.
- /somador_uvm

Referência

- https://www.accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.2.pdf
- https://verificationacademy.com/cookbook/uvm-universal-verification-methodology/

Página em branco.